Open Access | https://doi.org/10.25080/khdg8264



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Voice Capture, Structured Extraction, and Multi-Source Routing: Building and Evaluating a Field-Inspection Assistant in Python

Carolyn Olsen[1]

[1]Independent Researcher

Abstract

Field workers in agriculture, ecology, equipment inspection, and clinical settings record observations with their hands full and their attention on the task, then need to combine what they recorded with authoritative reference knowledge to decide what to do next. We present HiveGuide, an open-source Python application built for beekeeping that captures spoken hive-inspection observations, extracts them into a typed schema, and answers questions using a multi-source retrieval-augmented generation (RAG) assistant drawing on both the beekeeper's own structured records and an authoritative document corpus. Building the assistant surfaced an under-studied design problem, *source selection*: for each question, should the system query the structured database, the document corpus, or both? We share an empirical comparison of seven routing strategies across retrieval accuracy, response quality, latency, cost, and reliability on 501 labeled queries, used to inform the app's routing strategy. No single strategy wins: a supervised classifier maximizes accuracy (97.8%) but needs labeled data; an embedding similarity router reaches 87% with no training; heuristics are cheapest and fastest; a large language model (LLM) classifier produces the most grounded answers; and agentic routing retrieves the most relevant context but fails on roughly half of queries. We distill these trade-offs into a deployment-priority decision framework. HiveGuide is a beekeeping application as-is, but its components are domain-agnostic and it is meant to be forked: practitioners in agriculture, field science, and beyond can swap in their own schema, corpus, and routing keys.

Keywords retrieval-augmented generation, multi-source RAG, query routing, structured extraction, scientific software, FastAPI, pgvector, field inspection, beekeeping

Published Jul 13, 2026

Correspondence to
Carolyn Olsen
carolyn.olsen@gmail.com

Open Access

Copyright © 2026 Olsen. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

A beekeeper inspecting a hive is holding a frame covered in two thousand bees, wearing propolis-sticky gloves, and trying to remember what they will need to write down later: the weather, whether the queen was laying, how many frames held brood, the hive's weight, a few small hive beetles sighted, and what to do before the next visit. Voice memos capture audio but produce no queryable data; note apps require typing, later (if you remember). Neither yields structured records that can be analyzed or compared against expert guidance.

This constraint is not specific to beekeeping. Crop scouts, ecological surveyors, HVAC and wind-turbine technicians, and clinicians on rounds all work with their hands occupied and limited ability to interact with a device. Their decisions depend on *both* their own observation history *and* domain expertise, and they all need output that is structured for later

analysis. Generic tools (voice memos, note apps, general-purpose chat assistants) miss these constraints.

We built **HiveGuide**, an open-source Python application that targets this setting directly: it captures spoken observations, extracts them into a typed schema, and provides an assistant that answers questions by combining the user's structured records with an authoritative document corpus. We call this combination (voice capture, structured extraction, and multi-source RAG) the **capture-extract-retrieve pattern**. Beekeeping is our case study because source credibility matters (extension-service guidance, not arbitrary web text) and because answering a question such as *"Is my hive's weight normal for October in Wisconsin?"* genuinely requires both personal data (what did my hives weigh?) and reference knowledge (what should a hive weigh going into winter?).

Building the assistant surfaced a problem that turns out to be the crux of multi-source RAG: **source selection**. Given a question, should the system query the structured database, the document corpus, or both? This is distinct from two well-studied problems: *model selection* (which LLM to generate with) and *adaptive retrieval* (whether to retrieve at all). It is comparatively under-evaluated, and we therefore treat source selection as a research question in its own right and compare seven routing strategies in a single controlled, reproducible evaluation.

The study, though, is in service of a larger goal. We release HiveGuide as a **forkable template** for field-inspection assistants: a practitioner in agriculture, ecology, or equipment inspection can adopt the capture-extract-retrieve pattern wholesale, swap in their own schema and corpus, and use our results to pick a router for their own data. A forker thus inherits both the code and a decision procedure: the application demonstrates the capture-extract-retrieve pattern, and the study that follows is the evidence behind its single hardest design decision.

Contributions.

1. **HiveGuide**, an open-source Python application demonstrating the capture-extract-retrieve pattern for field inspection, built on FastAPI, PostgreSQL/pgvector, and LangChain.
2. A design pattern for **reliable LLM extraction** into a typed schema using Pydantic constraints, optional-by-default semantics, and reject-and-retry.
3. An **empirical comparison of seven source-selection strategies** across accuracy, quality, latency, cost, and reliability, with the evaluation harness, the 501 queries, and the frozen results released for reproducibility.
4. A **deployment-priority decision framework** and a domain-agnostic template others can port to their own schema, corpus, and routing keys.

2. Related Work

Multi-source RAG systems increasingly operate over heterogeneous backends: SQL databases, document corpora, knowledge graphs, web search. Within this space, *source selection* (which backend to query) is distinct from *model selection* (which LLM to use) and *adaptive retrieval* (whether to retrieve), and it is the least systematically evaluated of the three.

Recent multi-source architectures emphasize *coordination* rather than comparison of routing mechanisms. HM-RAG [1] decomposes queries across modality-specific retrievers and votes over results; HydraRAG [2] adds an agentic source selector with cross-source verification; RAS [3] builds query-specific knowledge graphs. These systems use LLM agents to select sources but do not benchmark alternative routing approaches.

The routing literature that *does* compare strategies largely targets model selection. RouteLLM [4] and Hybrid LLM [5] train routers to send queries to a strong or weak model, and RAGRouter [6] routes among retrieval-augmented models, but none addresses which *data source* to query. Work on adaptive retrieval (Adaptive-RAG [7] and Self-RAG [8]) asks *whether* to retrieve; its finding that universal retrieval underperforms adaptive strategies motivates our always-both baseline.

Agentic approaches treat retrieval as dynamic tool use: the ReAct loop [9], corrective retrieval in CRAG [10], and the patterns catalogued in a recent agentic-RAG survey [11]. This work explores agent *design* rather than comparing agents against non-agentic routers, which our two agentic strategies do directly. Finally, RAG evaluation has matured through RAGAS [12] and related frameworks [13], [14], [15], [16], though latency and cost remain under-reported relative to quality [17]. We adopt RAGAS quality metrics and add explicit latency and cost, comparing seven source-selection strategies in a single controlled setting.

3. The HiveGuide system

HiveGuide has three components (Figure 1): voice **capture** with structured extraction, **ingestion** of an authoritative corpus, and a multi-source **assistant**. The backend is FastAPI [18] on PostgreSQL [19] with the pgvector extension [20]; orchestration uses LangChain [21]; the client is a React Native (iOS) app; and all model calls are routed through OpenRouter: GPT-4o for intent classification, `gpt-oss-120b` for generation, and `text-embedding-3-large` for vectors. These three lanes are the units a forker keeps; only their domain-specific contents change from one field to the next (Table 1).

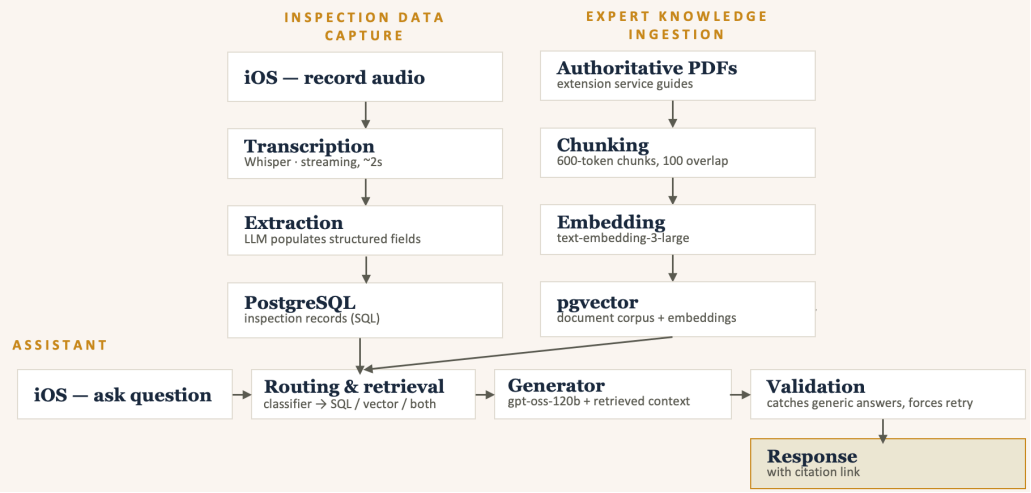


Figure 1. *HiveGuide architecture. The* capture *lane streams audio from iOS, transcribes it (∼2 s latency), and extracts typed fields into PostgreSQL. The* ingestion *lane chunks authoritative PDFs and embeds them into pgvector. The* assistant *lane classifies each question, routes it to the structured store, the vector store, or both, and generates a cited answer with* `gpt-oss-120b`.

Capture. The iOS client streams audio over a WebSocket in two-second chunks (48 kHz, 16-bit mono) and receives incremental transcriptions from OpenAI Whisper (`whisper-1`) at roughly two seconds of latency. The completed transcription is then converted into a typed inspection record (next section). Each record is stored in PostgreSQL with a hive reference, timestamp, the raw transcription and notes, the structured observation fields (weather, temperature, queen/eggs/larvae/capped-brood visibility, laying pattern, activity level), and extracted action items.

Ingestion. The corpus is eight authoritative beekeeping references from cooperative-extension services and similar bodies, for example the Virginia Cooperative Extension varroa and small-hive-beetle guides, a Maine Department of Agriculture fall-management guide, and a Center for Rural Affairs seasonal guide. Sources were chosen so their licenses permit reuse (Creative Commons Attribution, public domain, or explicit permission), which we record alongside each document. Documents are split into overlapping chunks (600/100, via LangChain's `RecursiveCharacterTextSplitter`), embedded with `text-embedding-3-large` (3072 dimensions), and stored in pgvector.

Assistant. A question is classified into an intent, routed to the structured store, the vector store, or both, and answered by `gpt-oss-120b` over the retrieved context with a citation link back to the source document. Personal data is fetched through a `get_user_hive_data` tool that queries the user's own hives over a configurable window (default one year) and formats the matching inspections as context; document retrieval ranks chunks by cosine similarity and returns the top ten (displaying up to five sources). A lightweight validation step rejects generic, ungrounded answers and forces a retry.

Work required to fork for other disciplines: The pipelines and the routing interface can run unchanged. Forkers will need to swap out the schema, the corpus, and the routing keys, as well as retouch a short list of tools and labels: the SQL query, the heuristic cues, the hive as the inspected unit, so they point at the new domain (Table 1).

Table 1. *What a forker keeps, swaps, and retouches.*

Adaptation	Components
Keep as-is	Capture, ingestion, and assistant lanes; routing to the structured store, the vector store, or both; validation harness
Swap	Extraction schema (change from queen/eggs/brood to relevant inspection fields); document corpus (change from extension PDFs to relevant authoritative sources); routing keys (labeled example queries for the embedding or supervised router)
Retouch	Personal-data SQL tool (`get_user_hive_data`); heuristic cues and LLM few-shot examples; hive entity model; rule-based extraction fallback; iOS labels and pickers

4. Reliable structured extraction

The value of voice capture depends on extraction producing data that downstream queries can trust. We enforce this with a Pydantic [22] schema rather than free-form JSON. The request model declares typed fields, closed vocabularies as regex constraints, and, crucially, every observation field as `Optional`:

```
class InspectionCreateRequest(BaseModel):
    hive_id: int
    transcription: str = ""
    notes: str = ""
    weather: Optional[str] = Field(
        None, pattern="^(sunny|cloudy|partly_cloudy|rainy|snowy)$")
    temperature: str = ""
    queen_visible: Optional[bool] = None
    eggs_visible: Optional[bool] = None
    larvae_visible: Optional[bool] = None
    capped_brood_visible: Optional[bool] = None
    laying_pattern: Optional[str] = Field(
        None, pattern="^(poor|patchy|solid)$")
    activity_level: Optional[str] = Field(
        None, pattern="^(low|average|high)$")
```

Three decisions make the output reliable. **Closed vocabularies as constraints**: `laying_pattern` must be `poor`, `patchy`, or `solid`, so the model cannot invent a value and

downstream queries need not fuzzy-match. **Optional by default**: "I didn't see the queen" is not the same as "the queen is absent," so unobserved fields are `None` rather than `False`; preserving nulls keeps later analysis honest. **Reject-and-retry**: the Pydantic constraints reject any value outside its enumerated vocabulary at the API boundary, and the LLM extraction retries up to twice on malformed output before falling back to a rule-based parser, so malformed extractions never reach the database.

As an illustration, the spoken note *"About 65 and cloudy. Didn't spot the queen, but there are eggs and a nice solid brood pattern, lots of bees. I need to check for varroa next time."* extracts to `weather="cloudy"`, `temperature="65"`, `eggs_visible=True`, `laying_pattern="solid"`, `activity_level="high"`, and an action item *"check for varroa"*, while `queen_visible` remains `None`, because the beekeeper said they did not *see* the queen, not that the queen is absent. This pattern (typed schema, optional semantics, validation-driven retry) is the reusable core of the capture pipeline and is independent of the beekeeping vocabulary. It is also what turns spoken notes into analyzable data: each inspection lands as a typed, timestamped row that ordinary SQL or dataframe tools can query directly, rather than as text to be parsed later.

5. The routing problem

Once inspections are structured and a corpus is indexed, the assistant faces two sources of truth. Some questions need only **personal** data from PostgreSQL ("Which of my hives are underweight?"); some need only **documents** from pgvector ("What are signs of varroa?"); and some need **both** ("Is my hive's weight normal for October in Wisconsin?"), comparing personal data against authoritative norms. This is the situation of any scientist whose measurements live in a database and whose domain literature lives in PDFs. We frame source selection as classifying each query into one of three intents (`personal_only`, `documents_only`, or `both_combined`) and routing accordingly.

Critically, *all* classification-based strategies share a single routing-and-generation pipeline; they differ only in how they produce the intent label. This makes the comparison a controlled experiment: the retrieval and generation code is held constant.

```
def query_with_simple_llm(self, question, user_id, routing, ...):
    user_data_text = None
    if routing in ("personal_only", "both_combined"):
        user_data_text = UserHiveDataTool(user_id)._run()      # SQL

    document_context = None
    if routing in ("documents_only", "both_combined"):
        document_context = self._run_retrieval(question)       # pgvector

    # Build a prompt from whichever contexts were retrieved,
    # then generate the answer with gpt-oss-120b.
    return self._generate(question, user_data_text, document_context)
```

As a concrete example, the question *"What are small hive beetles and where did they originally come from?"* is classified `documents_only`; the assistant queries only pgvector, retrieves the Virginia Cooperative Extension *Small Hive Beetle* guide (cosine similarity 0.70) as the top chunk, and `gpt-oss-120b` answers that the beetle (*Aethina tumida*) is native to sub-Saharan Africa, with a citation link back to that document. By contrast, *"Is my hive's weight normal for October in Wisconsin?"* routes to `both_combined`, drawing the hive's recorded weights from PostgreSQL and seasonal norms from the corpus before generating a single grounded answer.

6. Routing strategies

We compare seven strategies for producing the intent label.

- **Heuristic.** Hand-coded keyword rules (Table 2): personal cues such as “my” or “which of my” route to `personal_only`; reference cues such as “what are” or “how to” route to `documents_only`; ambiguous queries default to `both_combined`.
- **Embedding similarity.** A RouteLLM-style k-NN router (shown below) over 20 labeled example queries per category (60 vectors) drawn from the training split, using `k=3` neighbors tuned on a 99-query held-out split (20% of the 499-query training split, stratified by intent, seed 42). The 501-query evaluation set is not used to choose `k`.
- **Supervised.** A LightGBM [23] classifier (`num_leaves=31`, `learning_rate=0.05`, 100 boosting rounds) trained on TF-IDF features (unigrams and bigrams, up to 5000, via scikit-learn [24]) of the 499-query training split with a stratified 75/25 split (`train_classifier.ipynb`; artifacts in `models/`).
- **LLM classifier.** GPT-4o with few-shot prompting; HiveGuide’s deployed default. Label log-probabilities are recorded as a confidence score for evaluation but are not used for routing.
- **Agent discretion.** A ReAct agent with SQL and vector-search tools, free to decide which to call (max 10 iterations).
- **Agent + intent tool.** The same agent, additionally able to call a `classify_user_intent` tool when uncertain (invoked on 12% of queries).
- **Always-both.** A baseline that queries both sources for every question.

Table 2. *Heuristic routing rules (abbreviated).*

Cue words in query	Routed intent
“my”, “our”, “I”, “which of my”	personal_only
“what are”, “how to”, “signs of”	documents_only
mixed or ambiguous	both_combined

The embedding router embeds the incoming query and, for each intent category, averages the top-k cosine similarities (computed with NumPy [25]) to that category’s labeled examples (the RouteLLM approach of using k neighbors rather than a single nearest match), then routes to the highest-scoring category:

```
def classify_intent(self, query, k=None):
    if k is None:
        k = self.default_k                          # k=3, tuned on 99-query held-out split
    query_emb = self.embed_query(query)
    category_scores = {}
    for category, data in self.index["categories"].items():
        sims = np.array([
            self.cosine_similarity(query_emb, ex_emb)
            for ex_emb in data["embeddings"]
        ])
        top_k = np.sort(sims)[-k:]                  # top-k per category
        category_scores[category] = np.mean(top_k)
    return max(category_scores, key=category_scores.get)
```

All four classifier strategies share the same generation step (`gpt-oss-120b`, temperature 0.3); the two **agentic** strategies instead use a LangChain tool-calling agent (`max_iterations=10`) over the `get_user_hive_data` and `search_beekeeping_documents` tools.

7. Evaluation methodology

Dataset. We seed a test apiary of 4 hives with 26 inspection records spread over roughly eight months, and index the 8-source document corpus. Queries were synthesized with an LLM (`gpt-oss-120b`) along a path that fixes each query's ground-truth intent by construction: `documents_only` queries are generated from sampled PDF chunks, `personal_only` queries from the SQL schema, and `both_combined` queries from few-shot examples. The 1,000 generated queries were split, stratified by intent, into a 499-query **training split** (used to train the supervised classifier and to supply the embedding router's example queries) and a 501-query **test split** (167 per intent category) that serves as the evaluation set; no query row appears in both splits, though some question texts are duplicated across them (see Limitations). Each query record carries its intent, the expected source documents, and, where applicable, the specific ground-truth chunk(s) it should retrieve (encoded as `filename#chunk_position`).

Metrics. Retrieval **Hit@3/Hit@5** counts a query as a hit when at least one of its annotated ground-truth chunks appears among the top-k retrieved chunks; retrieval ranks the corpus by cosine similarity and returns the top ten. Queries without chunk-level ground truth (most `personal_only` queries) are excluded from the hit-rate denominator. Response quality uses two **RAGAS** metrics (context relevance and response groundedness, 0-1) judged reference-free by Claude Haiku 4.5 at temperature 0; context relevance is computed only for queries that retrieved document context. We use Haiku because it is inexpensive and because it comes from a different model family than the GPT-4o classifier and the `gpt-oss-120b` generator, so the judge is not scoring outputs from its own family. End-to-end **latency** is measured with `perf_counter` and includes retries, and **cost** combines per-model token usage at list prices for GPT-4o, `gpt-oss-120b`, and the Claude Haiku 4.5 judge. A query counts as an **error** when the strategy returns no usable answer after retries: the fallback message, an agent max-iterations stop, or an exception.

Run conditions. All strategies share the same generation model (`gpt-oss-120b`) and run over a four-worker thread pool; query sampling uses a fixed seed (42); each query allows up to two retries on an empty or failed response; and results are checkpointed so a run can resume.

Reproduction. The harness exposes each strategy through a small `ValidationStrategy` interface, so new strategies subclass it and register in a registry. The full comparison runs from three commands:

```
# 1. Seed a test apiary (4 hives, 26 inspections)
python -m validation.seed_validation_data --email you@example.com --password ...
# 2. Compare strategies on the committed evaluation queries (the 501-query test split)
python validation/run_strategy_validation.py \
    --queries-file validation/queries/questions_ground_truth_test.csv \
    --strategy heuristic_classifier --strategy supervised_classifier \
    --email you@example.com
```

To build a fresh query set for a new domain instead, `validation/generate_queries.py` regenerates the query pool with ground truth. Re-running the full evaluation requires API keys; for readers who only want the numbers, frozen per-strategy results and the summary table are committed under `validation/results/final/`.

8. Results

Figure 2 plots the central trade-off (Hit@5 against p95 latency), and Table 3 and Table 4 give the full picture. Absolute hit rates are low by construction and should be read comparatively across strategies (see Limitations). The headline is that **no single strategy dominates**.

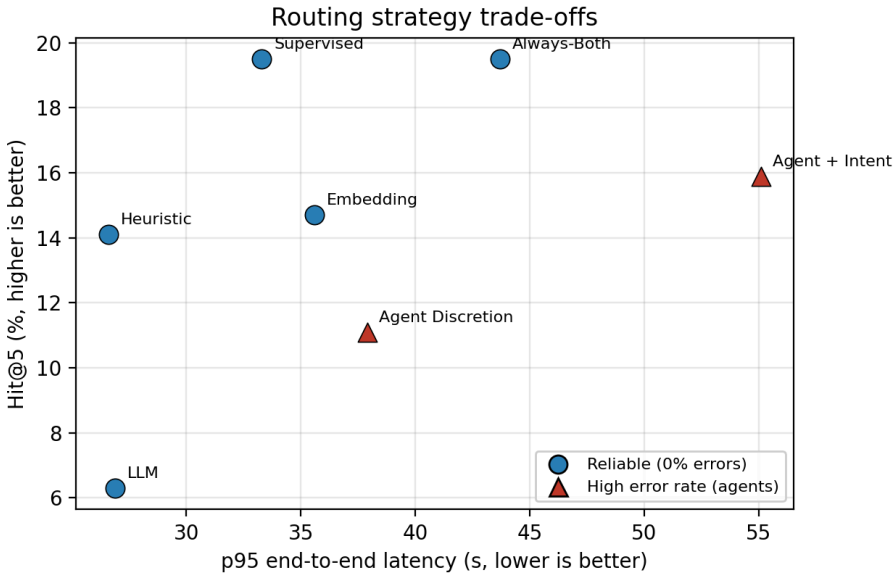


Figure 2. *Routing strategy trade-offs. Reliable classification strategies (circles) cluster at low latency; the supervised classifier matches the always-both baseline on Hit@5 at lower latency. Agentic strategies (triangles) are slower and, more importantly, fail on a large fraction of queries.*

Table 3. *Summary metrics across 501 queries. Cost is the total for one full evaluation run (501 queries). An error is a query with no usable answer (fallback message, max-iterations stop, or exception). Bold marks the best value per column among the zero-error strategies.*

Strategy	Hit@5 %	Class. Acc. %	p95 s	Run cost USD	Error %
Heuristic	14.1	73.1	**26.6**	**1.39**	0.0
LLM	6.3	54.9	26.9	1.54	0.0
Supervised	**19.5**	**97.8**	33.3	1.46	0.0
Embedding	14.7	87.0	35.6	1.44	0.0
Agent Discretion	11.1	n/a	37.9	1.27	54.7
Agent + Intent	15.9	n/a	55.1	1.70	49.3
Always-Both	**19.5**	n/a	43.7	1.71	0.0

Table 4. *Response quality (RAGAS, 0-1, higher is better; bold marks the best value per column). Groundedness averages over all 501 queries, with failed queries scoring 0 (success-only agent values appear in the text). Context relevance is computed only for queries that retrieved document context.*

Strategy	Context relevance	Groundedness
Heuristic	0.417	0.524
LLM	0.363	**0.648**
Supervised	0.472	0.513
Embedding	0.390	0.511
Agent Discretion	0.588	0.106
Agent + Intent	**0.723**	0.142
Always-Both	0.388	0.583

Classification accuracy tracks retrieval. The supervised classifier reached 97.8% accuracy and the best Hit@5 (19.5%), matching always-both while avoiding unnecessary retrieval. The embedding router achieved 87.0% with no model training (only labeled

example queries), making it attractive when training data is scarce. Heuristics captured surprising signal (73.1%) from keyword patterns alone. The two numbers measure different steps: routing accuracy is whether we queried the right source; Hit@5 is whether the vector store then returned the annotated chunk. Supervised routing is nearly perfect and still ties always-both at 19.5%, so the ceiling is retrieval, not source selection.

The LLM classifier underperformed at routing. Despite semantic capability, it scored only 54.9% accuracy, with just 11.9% recall on `documents_only`: it was biased toward `personal` and `combined` and missed pure-document questions, which explains its low Hit@5 (6.3%). Yet its *answers* were the most grounded (0.648): when it did retrieve, responses stayed close to the evidence. That groundedness, together with its zero error rate, is why it remains HiveGuide's deployed default despite its routing accuracy.

Agents trade reliability for context quality. The two agentic strategies achieved the highest context relevance (0.723 and 0.588), but over small, survivor-biased denominators: relevance is scored only for queries that both succeeded and retrieved document context (119 and 114 queries, respectively). Their groundedness in Table 4 (0.142 and 0.106) averages over all 501 queries with failures scored 0; restricted to successful queries, groundedness rises to 0.280 and 0.233, still the lowest of any strategy. In short, the agents fetched relevant context but failed outright on roughly half of queries (49.3% and 54.7%), and even their successful answers were the least supported by retrieved evidence. This points to a fix (separating tool-selection from a constrained response-drafting step) at a latency and cost penalty.

Evaluation cost dominates system cost. RAGAS judging accounted for roughly 70-96% of the $1.27-$1.71 total cost of each strategy's evaluation run; the system itself cost only $0.06-$0.51 per 501-query run, a fraction of a cent per query. For production monitoring, sampling 1-10% of queries would cut evaluation cost by one to two orders of magnitude.

From these results we distill a deployment-priority decision framework (Table 5). It is the practical payoff for a forker: source selection is the one knob the template leaves open, and the right setting depends on your latency budget and whether you have labeled data, not on the domain itself.

Table 5. *Deployment-priority decision framework.*

If you optimize for	Use	Caveat
Speed and cost	Heuristic	Lowest routing accuracy among reliable strategies (73.1%)
Maximum accuracy	Supervised classifier	Requires labeled queries
Semantics without training	Embedding router	Needs ∼20 labeled examples per intent
Grounded, traceable answers (HiveGuide's deployed choice)	LLM classifier	Weak at routing documents-only questions
Complex multi-step queries	Agentic routing	Fails on roughly half of queries (see Limitations)

9. Limitations

Absolute Hit@5 values are low (6-19%) because ground truth requires retrieving *specific* annotated chunks from a small corpus under strict top-k matching; the metric is best read comparatively across strategies rather than as end-task accuracy.

Agent failures shape the agent quality metrics. Under our error definition (no usable answer after retries), agent discretion failed on 54.7% of queries and agent + intent on 49.3%,

mostly from exhausting the 10-iteration budget on a mid-tier open-source model and from tool-response parsing errors. Context relevance is consequently computed over a smaller, survivor-biased sample, and the groundedness means in Table 4 include failed queries scored 0 (success-only values appear in the Results). With a stronger model, better prompt tuning, and more engineering, the agents, and the LLM classifier's routing accuracy, could improve; for a single-developer side project, the LLM classifier was good enough.

The evaluation and training queries were generated together and split by row rather than by text: no query row appears in both splits, but the LLM generator produced duplicate question texts (concentrated in the documents-only category), so about 30% of evaluation questions share their exact text with a training-split question. This affects only the two learned routers; the other strategies, and all retrieval, quality, latency, and cost metrics, involve no training on queries. As a robustness check, restricting to the 352 evaluation queries whose text does not appear in the training split (macro-averaging accuracy over intents, since the duplicates are unevenly distributed) leaves the supervised classifier essentially unchanged (97.7% vs. 97.8%) and lowers the embedding router modestly (84.3% vs. 87.0%), so the duplication does not drive our conclusions.

The reported numbers also come from a single pass over the 501 queries rather than averaged repeats, so small differences between adjacent strategies should not be overinterpreted; the large gaps that drive our conclusions (for example supervised vs. LLM routing accuracy, or agent error rates) are well outside plausible run-to-run noise. Finally, results come from one domain and depend on hosted model APIs; the frozen artifacts allow inspection without rerun, and the harness is designed to be re-run on other domains.

10. Conclusion: fork it

Building a practical field-inspection assistant led us to a research question: when a system has both structured and unstructured sources, *where* to query is a design decision distinct from model selection and from whether to retrieve. Across seven strategies there is no universal winner, only strategies that fit particular deployment priorities, and in consequential domains, traceable citations are how a RAG system earns the right to give advice.

That study exists to support an artifact we want others to use. The capture-extract-retrieve pattern is not specific to beekeeping: it fits any setting where a practitioner records observations in the field and must weigh them against authoritative reference knowledge. Adapting HiveGuide is the split in Table 1: swap your **schema** (replace queen/eggs/brood with the fields your inspections capture), your **corpus** (swap extension PDFs for your authoritative sources), and your **routing keys** (relabel example queries for the embedding router or retrain the supervised classifier), and retouch the few tools and labels that still say hive. The capture, ingestion, and assistant lanes and the evaluation harness stay the same. So we release the entire open-source application together with the harness and the queries and frozen results behind this study, not just the headline numbers. Fork HiveGuide, swap in your schema, corpus, and routing keys, and ship it for your own field.

References

[1] P. Liu *et al.*, "HM-RAG: Hierarchical Multi-Agent Multimodal Retrieval Augmented Generation," in *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, Dublin, Ireland, 2025. doi: 10.1145/3746027.3754761.

[2] X. Tan *et al.*, "HydraRAG: Structured Cross-Source Enhanced Large Language Model Reasoning," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*, Suzhou, China, 2025, pp. 14442–14470. doi: 10.18653/v1/2025.emnlp-main.730.

[3] P. Jiang *et al.*, "RAS: Retrieval-And-Structuring for Knowledge-Intensive LLM Generation," *arXiv preprint*, 2025.

[4] I. Ong *et al.*, "RouteLLM: Learning to Route LLMs with Preference Data," in *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[5] D. Ding *et al.*, "Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[6] J. Zhang, X. Liu, Y. Hu, C. Niu, F. Wu, and G. Chen, "RAGRouter: Learning to Route Queries to Multiple Retrieval-Augmented Language Models," in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

[7] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, Mexico City, Mexico, 2024, pp. 7036–7050. doi: 10.18653/v1/2024.naacl-long.389.

[8] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[9] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023.

[10] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," *arXiv preprint*, 2024.

[11] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," *arXiv preprint*, 2025.

[12] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL 2024)*, St. Julians, Malta, 2024, pp. 150–158. doi: 10.18653/v1/2024.eacl-demo.16.

[13] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, Mexico City, Mexico, 2024, pp. 338–354. doi: 10.18653/v1/2024.naacl-long.20.

[14] T. Ding, A. Banerjee, L. Mombaerts, Y. Li, T. Borogovac, and J. P. De la Cruz Weinstein, "VERA: Validation and Evaluation of Retrieval-Augmented Systems," *arXiv preprint*, 2024.

[15] A. Salemi and H. Zamani, "Evaluating Retrieval Quality in Retrieval-Augmented Generation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*, Washington, DC, USA, 2024, pp. 2395–2400. doi: 10.1145/3626772.3657957.

[16] R. Friel, M. Belyi, and A. Sanyal, "RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems," *arXiv preprint*, 2024.

[17] A. Gan *et al.*, "Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey," *arXiv preprint*, 2025.

[18] S. Ramírez, "FastAPI." [Online]. Available: https://github.com/fastapi/fastapi

[19] PostgreSQL Global Development Group, "PostgreSQL." [Online]. Available: https://www.postgresql.org/

[20] A. Kane, "pgvector: Open-source vector similarity search for PostgreSQL." [Online]. Available: https://github.com/pgvector/pgvector

[21] H. Chase, "LangChain." [Online]. Available: https://github.com/langchain-ai/langchain

[22] S. Colvin and Pydantic contributors, "Pydantic: Data validation using Python type hints." [Online]. Available: https://github.com/pydantic/pydantic

[23] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[24] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[25] C. R. Harris, K. J. Millman, S. J. van der Walt, and others, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.